

补充模拟卷 01：自顶向下分析与 LR 概念补缺

A 卷题目

一、递归下降、回溯与左公因子（20 分）

文法：

$$\begin{aligned} S &\rightarrow a A \mid a B \\ A &\rightarrow c \\ B &\rightarrow d \end{aligned}$$

1. 说明该文法在递归下降分析中为什么可能需要回溯。
2. 提取左公因子。
3. 判断提取左公因子后的文法是否适合构造 LL(1) 预测分析表。

二、二义性文法与预测分析表（20 分）

文法：

$$E \rightarrow E + E \mid E * E \mid id$$

1. 说明该文法为什么不适合直接构造 LL(1) 预测分析表。
2. 指出它至少存在的两个问题。
3. 简述通常需要怎样处理。

三、句柄与归约序列（20 分）

文法：

$$\begin{aligned} S &\rightarrow A B \\ A &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

对输入串：

a b

1. 写出自底向上的归约序列。
2. 说明该归约序列与最右推导的关系。

四、规约-规约冲突（20 分）

某 LR(0) 状态中有：

$$\begin{aligned} A &\rightarrow id \cdot \\ B &\rightarrow id \cdot \end{aligned}$$

1. 说明为什么这是规约-规约冲突。
2. 若 $FOLLOW(A) = \{ ; \}$, $FOLLOW(B) = \{) \}$, 说明 SLR 如何处理。
3. 若 $FOLLOW(A)$ 和 $FOLLOW(B)$ 有公共符号, 会发生什么。

五、内核项、非内核项与 SLR 弱点（20 分）

回答：

1. 什么是 LR(0) 项集中的内核项。
2. 什么是非内核项。
3. SLR 为什么比 LR(0) 强。
4. SLR 为什么仍然有弱点。

B 按考试标准重写答案

一、递归下降、回溯与左公因子答案

1. 为什么可能需要回溯

原文法中 S 有两条产生式：

$$\begin{aligned} S &\rightarrow a A \\ S &\rightarrow a B \end{aligned}$$

递归下降分析在选择产生式时，通常先看当前输入符号。若输入串以 a 开头，则两条产生式的第一个终结符都是 a ：

产生式	可见的第一个输入符号
$S \rightarrow a A$	a
$S \rightarrow a B$	a

因此只看 1 个输入符号时无法确定应选择哪条产生式。若分析器先选 $S \rightarrow a A$ ，但后续输入是 $a d$ ，则 $A \rightarrow c$ 无法匹配 d ，必须退回去改选 $S \rightarrow a B$ 。这就是回溯来源。

2. 提取左公因子

把公共前缀 a 提出来：

$$\begin{aligned} S &\rightarrow a S' \\ S' &\rightarrow A \mid B \\ A &\rightarrow c \\ B &\rightarrow d \end{aligned}$$

由于 $A \rightarrow c$ 、 $B \rightarrow d$ ，也可以等价写成：

$$\begin{aligned} S &\rightarrow a S' \\ S' &\rightarrow c \mid d \end{aligned}$$

3. LL(1) 判断

对提取左公因子后的文法计算 FIRST：

非终结符或右部	FIRST
c	$\{ c \}$
d	$\{ d \}$
$S' \rightarrow c$	$\{ c \}$
$S' \rightarrow d$	$\{ d \}$
$S \rightarrow a S'$	$\{ a \}$

S' 的两条产生式 FIRST 集不相交：

$$\text{FIRST}(c) \cap \text{FIRST}(d) = \text{empty}$$

因此预测分析表中不会在同一表项填入两条产生式。对应表项为：

非终结符	输入 a	输入 c	输入 d	输入 $\$$
S	$S \rightarrow a S'$	error	error	error
S'	error	$S' \rightarrow c$	$S' \rightarrow d$	error

所以提取左公因子后的文法适合构造 LL(1) 预测分析表。

二、二义性文法与预测分析表答案

1. 为什么不能直接构造 LL(1) 表

文法：

$E \rightarrow E + E \mid E * E \mid id$

不能直接构造 LL(1) 预测分析表，原因有两个层次。

第一，它有直接左递归：

$E \rightarrow E + E$
 $E \rightarrow E * E$

递归下降分析 E 时会立即再次调用 E，可能无限递归。

第二，它没有规定 +、* 的优先级和结合性，因此是二义性的。例如输入：

$id + id * id$

可以有两种结构：

分析结构	含义
$(id + id) * id$	先算加法，再算乘法
$id + (id * id)$	先算乘法，再算加法

同一个输入串有不同语法树，所以不适合直接做确定的预测分析。

2. 至少存在的两个问题

问题	具体表现	对 LL(1) 的影响
直接左递归	$E \rightarrow E + E, E \rightarrow E * E$	递归下降会无限递归
二义性	没有体现优先级和结合性	同一个输入串有多种语法树
FIRST 冲突	三条产生式都可能以 id 开始	预测分析表的同一格会出现多条产生式

考试中写出前两个问题即可完整回答第 2 问；写出 FIRST 冲突可以补强说明。

3. 通常处理方式

先改写成表达优先级和结合性的非二义文法，再消除左递归。常用改写为：

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

该文法体现了 * 的优先级高于 +。为了构造 LL(1) 表，再消除左递归：

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \epsilon$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \epsilon$
 $F \rightarrow id$

最后再计算 FIRST、FOLLOW 并填写预测分析表。

三、句柄与归约序列答案

1. 自底向上的归约序列

文法：

```
S -> A B
A -> a
B -> b
```

输入串是：

a b

自底向上归约时，每一步选择当前句型中的句柄，即某个产生式右部，然后把它归约为产生式左部：

当前句型	句柄	使用产生式	归约后
a b	a	A -> a	A b
A b	b	B -> b	A B
A B	A B	S -> A B	S

所以归约序列为：

```
a b
=> A b
=> A B
=> S
```

2. 与最右推导的关系

自底向上归约是最右推导的逆过程。上面归约序列反过来就是一个最右推导：

```
S
=> A B
=> A b
=> a b
```

检查“最右”的依据是：每次推导都替换当前句型中最右边的非终结符。

推导步骤	被替换的最右非终结符
S => A B	S
A B => A b	B
A b => a b	A

因此题目中的归约序列确实是该最右推导的逆序。

四、规约-规约冲突答案

1. 为什么是规约-规约冲突

某 LR(0) 状态中同时有两个完成项：

```
A -> id .
B -> id .
```

点在产生式末尾表示已经识别出完整右部，可以规约。在 LR(0) 中，完成项不看向输入符号，会尝试在所有可能输入上填规约动作。因此同一个状态会同时要求：

```
reduce A -> id
reduce B -> id
```

同一个 ACTION 表项不能同时放两个规约动作，所以这是规约-规约冲突。

2. SLR 如何处理给定 FOLLOW 集

SLR 的规则是：完成项 $A \rightarrow \alpha$ ，只在 $\text{FOLLOW}(A)$ 中的终结符上填规约。

题目给出：

$\text{FOLLOW}(A) = \{ ; \}$
 $\text{FOLLOW}(B) = \{) \}$

因此填表为：

输入符号	ACTION
;	$r(A \rightarrow id)$
)	$r(B \rightarrow id)$

因为 $\{ ; \}$ 与 $\{) \}$ 不相交，所以这两个规约动作不会落在同一个 ACTION 表项中，SLR 可以消除这个 LR(0) 状态中的冲突。

3. FOLLOW 有公共符号时的结果

若存在某个终结符 x ，使得：

$x \in \text{FOLLOW}(A)$
 $x \in \text{FOLLOW}(B)$

则 SLR 会同时填写：

$\text{ACTION}[\text{状态}, x] = r(A \rightarrow id)$
 $\text{ACTION}[\text{状态}, x] = r(B \rightarrow id)$

同一表项仍有两个规约动作，规约-规约冲突仍然存在。

五、内核项、非内核项与 SLR 弱点答案

1. 内核项

LR(0) 项集中的内核项包括两类：

1. 增广文法的初始项 $S' \rightarrow \cdot S$ 。
2. 所有点不在产生式最左端的 LR(0) 项。

例子：

项	是否内核项	原因
$S' \rightarrow \cdot S$	是	初始项特例
$A \rightarrow a \cdot B$	是	点不在最左端
$A \rightarrow a B \cdot$	是	点不在最左端

2. 非内核项

非内核项是除初始项以外，点在产生式最左端的项：

项	是否非内核项	原因
$A \rightarrow \cdot a B$	是	点在最左端，且不是 $S' \rightarrow \cdot S$
$B \rightarrow \cdot c$	是	点在最左端

这些项通常由 closure 运算自动补入。

3. SLR 为什么比 LR(0) 强

LR(0) 看到完成项 $A \rightarrow \alpha \cdot$ 后，不使用向前输入符号，规约范围过大。SLR 增加 FOLLOW 集限制：

LR(0) : $A \rightarrow \alpha \cdot$ 在所有终结符上规约
SLR : $A \rightarrow \alpha \cdot$ 只在 FOLLOW(A) 上规约

因此 SLR 可以消除一部分 LR(0) 中的移进-规约冲突和规约-规约冲突。

4. SLR 为什么仍有弱点

SLR 的 FOLLOW 集只与非终结符有关，不区分项目所在的具体 LR 状态和更精确的调用上下文。若 FOLLOW(A) 在全局范围内很大，SLR 会在某些当前状态不该规约的位置也填入规约动作。

对比：

方法	规约依据	精确程度
LR(0)	不看输入符号	最粗
SLR	看 FOLLOW(A)	比 LR(0) 精细，但 FOLLOW 是全局集合
LR(1)	看项目自身携带的向前看符号	更精确

因此有些文法不是 SLR(1)，但可以是 LR(1) 或 LALR(1)。